

```

--
=====
=====
--  GMAO - SCHEMA DE BASE DE DONNEES CIBLE
(PostgreSQL 14+)
--  Projet   : Application de Gestion de la
Maintenance Assistee par Ordinateur
--  Origine  : Migration / refonte de
BASE_DE_DONNEE_DE_GESTION_DE_LA_MAINTENANCE
.accdb
--  Auteur   : Jean Louis N'DRI - QHSE &
Planning
--  Version  : 1.0
--  Devise   : FCFA (XOF) - Fuseau :
Africa/Abidjan (UTC+0)
--  NOTE DEV : script de reference. A
adapter si un ORM (Prisma, Eloquent,
--              TypeORM, Django) genere les
migrations.
--
=====
=====

CREATE EXTENSION IF NOT EXISTS "uuid-ossf";
CREATE EXTENSION IF NOT EXISTS "unaccent";

--
=====
=====
-- 0. TYPES ENUMERES (referentiels fermes)

```

```

--
=====
=====
CREATE TYPE type_maintenance AS ENUM
('PREVENTIF','CORRECTIF','REGLEMENTAIRE','A
MELIORATIF','PREDICTIF');
CREATE TYPE statut_ot AS ENUM
('BROUILLON','PLANIFIE','EN_COURS','EN_ATT
ENTE','REALISE','CLOTURE','ANNULE');
CREATE TYPE priorite_ot AS ENUM
('P1_URGENT','P2_HAUTE','P3_NORMALE','P4_BA
SSE');
CREATE TYPE origine_ot AS ENUM
('DEMANDE','PLAN_PREVENTIF','CREATION_DIREC
TE','RONDE');
CREATE TYPE statut_di AS ENUM
('NOUVELLE','VALIDEE','REJETEE','CONVERTIE'
);
CREATE TYPE criticite_equip AS ENUM
('A','B','C');
CREATE TYPE statut_equip AS ENUM
('EN_SERVICE','A_L_ARRET','EN_PANNE','EN_RE
PARATION','REFORME');
CREATE TYPE type_mouvement AS ENUM
('ENTREE','SORTIE','RETOUR','AJUSTEMENT','I
NVENTAIRE');
CREATE TYPE type_periodicite AS ENUM
('JOUR','SEMAINE','MOIS','ANNEE','HEURE_FON
CTIONNEMENT','KILOMETRE','CYCLE');
CREATE TYPE statut_commande AS ENUM
('BROUILLON','ENVOYEE','PARTIELLEMENT_RECUE
','RECUE','ANNULEE');
--

```

```

=====
=====
-- 1. SOCLE : SECURITE, UTILISATEURS, SITES
--
=====
=====
CREATE TABLE role (
    id            SERIAL PRIMARY KEY,
    code          VARCHAR(30) NOT NULL
UNIQUE, -- ADMIN, RESP_MAINT, PLANIF,
TECH, MAGASIN, DEMANDEUR, DIRECTION, QHSE
    libelle       VARCHAR(80) NOT NULL,
    description    TEXT
);

CREATE TABLE permission (
    id            SERIAL PRIMARY KEY,
    code          VARCHAR(60) NOT NULL
UNIQUE, -- ex: ot.creer, ot.cloturer,
stock.sortir, param.gerer
    module        VARCHAR(40) NOT NULL,
    libelle       VARCHAR(120) NOT NULL
);

CREATE TABLE role_permission (
    role_id       INT NOT NULL REFERENCES
role(id) ON DELETE CASCADE,
    permission_id INT NOT NULL REFERENCES
permission(id) ON DELETE CASCADE,
    PRIMARY KEY (role_id, permission_id)
);

CREATE TABLE utilisateur (
    id            SERIAL PRIMARY KEY,

```

```

        email          VARCHAR(120) NOT NULL
        UNIQUE,
        mot_de_passe    VARCHAR(255) NOT
        NULL,           -- hash Argon2id ou bcrypt
        cost>=12
        nom             VARCHAR(80)  NOT NULL,
        prenom          VARCHAR(80),
        telephone       VARCHAR(30),
        role_id         INT NOT NULL REFERENCES
        role(id),
        site_id         INT,          -- FK
        ajoutée plus bas
        actif           BOOLEAN NOT NULL
        DEFAULT TRUE,
        mfa_actif       BOOLEAN NOT NULL
        DEFAULT FALSE,
        dernière_connexion TIMESTAMPTZ,
        doit_changer_mdp BOOLEAN NOT NULL
        DEFAULT TRUE,
        created_at      TIMESTAMPTZ NOT NULL
        DEFAULT now(),
        updated_at      TIMESTAMPTZ NOT NULL
        DEFAULT now(),
        deleted_at      TIMESTAMPTZ  --
        suppression logique (RG-12)
    );

    CREATE TABLE site (
        id              SERIAL PRIMARY KEY,
        code            VARCHAR(20) NOT NULL
        UNIQUE,        -- ex: ABJ, BSM, OUM
        libelle         VARCHAR(120) NOT NULL,

```

```

        client
    VARCHAR(120),                --
    TotalEnergies, OLA Energy, ELTON,
    PETROCI...
        adresse                TEXT,
        ville                  VARCHAR(80),
        pays                   VARCHAR(60) DEFAULT
    'Cote d'Ivoire',
        latitude                NUMERIC(10,7),
        longitude               NUMERIC(10,7),
        actif                   BOOLEAN NOT NULL
    DEFAULT TRUE
);
ALTER TABLE utilisateur ADD CONSTRAINT
fk_user_site FOREIGN KEY (site_id)
REFERENCES site(id);

CREATE TABLE localisation (
    id                          SERIAL PRIMARY KEY,
    site_id                     INT NOT NULL REFERENCES
site(id),
    parent_id                   INT REFERENCES
localisation(id),
    code                        VARCHAR(30) NOT NULL,
    libelle                     VARCHAR(120) NOT NULL,
    niveau                      SMALLINT DEFAULT
1,                            -- 1=zone, 2=atelier,
3=ligne, 4=poste
    UNIQUE (site_id, code)
);

--
=====
=====

```

```

-- 2. RESSOURCES HUMAINES TECHNIQUES
--
=====
=====
CREATE TABLE specialite (
    id                SERIAL PRIMARY KEY,
    code              VARCHAR(20) NOT NULL
UNIQUE,    -- MEC, ELEC, SOUD, HYD, INSTR,
FROID
    libelle          VARCHAR(80) NOT NULL
);

CREATE TABLE technicien (
    id                SERIAL PRIMARY KEY,
    matricule         VARCHAR(20) NOT NULL
UNIQUE,    -- reprise Access : S010, S011...
    nom_prenom       VARCHAR(150) NOT NULL,
    specialite_id    INT REFERENCES
specialite(id),
    responsable_id   INT REFERENCES
technicien(id), -- hierarchie interne
    utilisateur_id   INT REFERENCES
utilisateur(id), -- compte de connexion
(optionnel)
    site_id          INT REFERENCES
site(id),
    statut           VARCHAR(20) NOT NULL
DEFAULT 'ACTIF', -- ACTIF, CONGE, INACTIF,
SORTI
    cout_horaire     NUMERIC(12,2) DEFAULT
0,    -- FCFA / heure (valorisation MO)
    telephone       VARCHAR(30),
    date_embauche    DATE,
    habilitations

```

```

TEXT,                                --
electrique BR/B2V, travail en hauteur,
ATEX...
        created_at      TIMESTAMPTZ NOT NULL
DEFAULT now(),
        deleted_at      TIMESTAMPTZ
);
CREATE INDEX idx_tech_specialite ON
technicien(specialite_id);

--
=====
=====
-- 3. ACTIFS / PARC EQUIPEMENT
--
=====
=====
CREATE TABLE famille_equipement (
        id                SERIAL PRIMARY KEY,
        code              VARCHAR(20) NOT NULL
UNIQUE,    -- POM, COM, GRP, VEH, LEV, ELEC
        libelle          VARCHAR(120) NOT NULL
);

CREATE TABLE equipement (
        id                SERIAL PRIMARY KEY,
        code_equipement   VARCHAR(30) NOT
NULL UNIQUE,    -- RG-02 : SITE-FAM-NNN
        designation      VARCHAR(200) NOT
NULL,
        famille_id       INT REFERENCES
famille_equipement(id),
        parent_id        INT REFERENCES
equipement(id), -- arborescence sous-

```

```

ensembles
    localisation_id      INT REFERENCES
localisation(id),
    marque               VARCHAR(80),
    modele               VARCHAR(80),
    numero_serie         VARCHAR(80),
    fournisseur_id
INT,                      -- FK
ajoutee plus bas
    date_mise_service    DATE,
    valeur_acquisition
NUMERIC(14,2),           -- FCFA
    fin_garantie         DATE,
    criticite            criticite_equip NOT
NULL DEFAULT 'C',
    statut              statut_equip    NOT
NULL DEFAULT 'EN_SERVICE',
    unite_compteur
VARCHAR(20),             -- h, km,
cycles
    compteur_actuel      NUMERIC(14,2)
DEFAULT 0,
    qr_code              VARCHAR(60)
UNIQUE,                  -- etiquette terrain
    photo_url            TEXT,
    caracteristiques
JSONB,                   --
puissance, debit, tension... (souple)
    observations         TEXT,
    created_at           TIMESTAMPTZ NOT
NULL DEFAULT now(),
    updated_at           TIMESTAMPTZ NOT
NULL DEFAULT now(),
    deleted_at           TIMESTAMPTZ

```



```

);
CREATE INDEX idx_equip_localisation ON
equipement(localisation_id);
CREATE INDEX idx_equip_statut      ON
equipement(statut);
CREATE INDEX idx_equip_criticite   ON
equipement(criticite);

CREATE TABLE compteur_releve (
    id          SERIAL PRIMARY KEY,
    equipement_id INT NOT NULL REFERENCES
equipement(id) ON DELETE CASCADE,
    date_releve  DATE NOT NULL,
    valeur       NUMERIC(14,2) NOT NULL,
    unite        VARCHAR(20),
    releve_par   INT REFERENCES
utilisateur(id),
    created_at   TIMESTAMPTZ NOT NULL
DEFAULT now(),
    UNIQUE (equipement_id, date_releve)
);

--
=====
=====
-- 4. STOCK, ARTICLES, FOURNISSEURS
--
=====
=====
CREATE TABLE fournisseur (
    id          SERIAL PRIMARY KEY,
    code        VARCHAR(20) NOT NULL
UNIQUE,
    raison_sociale VARCHAR(150) NOT NULL,

```

```

        contact          VARCHAR(120),
        telephone        VARCHAR(30),
        email            VARCHAR(120),
        adresse          TEXT,
        delai_livraison_jours SMALLINT DEFAULT
0,
        note_evaluation SMALLINT CHECK
(note_evaluation BETWEEN 1 AND 5),
        actif           BOOLEAN NOT NULL
DEFAULT TRUE
    );
ALTER TABLE equipement ADD CONSTRAINT
fk_equip_fournisseur FOREIGN KEY
(fournisseur_id) REFERENCES
fournisseur(id);

CREATE TABLE categorie_article (
    id                SERIAL PRIMARY KEY,
    code              VARCHAR(20) NOT NULL
UNIQUE,
    libelle           VARCHAR(120) NOT NULL
);

CREATE TABLE article (
    id                SERIAL PRIMARY
KEY,
    ref_article        VARCHAR(40) NOT
NULL UNIQUE, -- corrige "Ref_Acticle" de
l'Access
    designation        VARCHAR(200) NOT
NULL,
    categorie_id        INT REFERENCES
categorie_article(id),
    marque             VARCHAR(80),

```

```

        unite                VARCHAR(20) NOT
NULL DEFAULT 'U', -- U, L, kg, m
        quantite_stock       NUMERIC(12,2)
NOT NULL DEFAULT 0 CHECK (quantite_stock >=
0),
        seuil_reappro        NUMERIC(12,2)
NOT NULL DEFAULT 0,
        stock_max            NUMERIC(12,2),
        prix_unitaire_moyen   NUMERIC(14,2)
NOT NULL DEFAULT 0, -- PUMP recalcule a
chaque entree
        emplacement_magasin   VARCHAR(40), --
allée/rayon/casier
        fournisseur_principal_id INT REFERENCES
fournisseur(id),
        delai_appro_jours     SMALLINT DEFAULT
0,
        garantie              VARCHAR(60),
        piece_critique         BOOLEAN NOT NULL
DEFAULT FALSE,
        actif                  BOOLEAN NOT NULL
DEFAULT TRUE,
        created_at             TIMESTAMPTZ NOT
NULL DEFAULT now(),
        deleted_at             TIMESTAMPTZ
);
CREATE INDEX idx_article_designation ON
article USING gin (to_tsvector('french',
designation));

-- Nomenclature : quelles pieces vont sur
quel équipement
CREATE TABLE equipement_article (

```

```

        equipement_id INT NOT NULL REFERENCES
equipement(id) ON DELETE CASCADE,
        article_id INT NOT NULL REFERENCES
article(id) ON DELETE CASCADE,
        quantite_montee NUMERIC(10,2) DEFAULT
1,
        PRIMARY KEY (equipement_id, article_id)
);

CREATE TABLE mouvement_stock (
        id BIGSERIAL PRIMARY KEY,
        article_id INT NOT NULL REFERENCES
article(id),
        type_mvt type_mouvement NOT
NULL,
        quantite NUMERIC(12,2) NOT NULL
CHECK (quantite > 0),
        prix_unitaire NUMERIC(14,2) NOT NULL
DEFAULT 0,
        montant NUMERIC(16,2)
GENERATED ALWAYS AS (quantite *
prix_unitaire) STORED,
        date_mvt TIMESTAMPTZ NOT NULL
DEFAULT now(),
        ot_id
INT,
        --
FK ajoutée plus bas
        commande_id INT,
        bon_reference VARCHAR(40),
        motif TEXT,
        stock_avant NUMERIC(12,2),
        stock_apres NUMERIC(12,2),
        utilisateur_id INT REFERENCES
utilisateur(id),

```

```

        created_at      TIMESTAMPTZ NOT NULL
DEFAULT now()
);
CREATE INDEX idx_mvt_article_date ON
mouvement_stock(article_id, date_mvt DESC);

CREATE TABLE commande_achat (
    id                  SERIAL PRIMARY KEY,
    numero              VARCHAR(30) NOT
NULL UNIQUE, -- BC-2026-00001
    fournisseur_id     INT NOT NULL
REFERENCES fournisseur(id),
    date_commande      DATE NOT NULL
DEFAULT CURRENT_DATE,
    date_livraison_prevue DATE,
    statut              statut_commande NOT
NULL DEFAULT 'BROUILLON',
    montant_total      NUMERIC(16,2)
DEFAULT 0,
    demandeur_id       INT REFERENCES
utilisateur(id),
    valide_par         INT REFERENCES
utilisateur(id),
    date_validation    TIMESTAMPTZ,
    observations       TEXT
);

CREATE TABLE commande_ligne (
    id                  SERIAL PRIMARY KEY,
    commande_id        INT NOT NULL REFERENCES
commande_achat(id) ON DELETE CASCADE,
    article_id         INT NOT NULL REFERENCES
article(id),
    quantite_cmd       NUMERIC(12,2) NOT NULL

```

```

CHECK (quantite_cmd > 0),
      quantite_recue NUMERIC(12,2) NOT NULL
DEFAULT 0,
      prix_unitaire NUMERIC(14,2) NOT NULL
DEFAULT 0
);
ALTER TABLE mouvement_stock ADD CONSTRAINT
fk_mvt_commande FOREIGN KEY (commande_id)
REFERENCES commande_achat(id);

--
=====
=====
-- 5. MAINTENANCE PREVENTIVE : GAMMES ET
PLANS
--
=====
=====
CREATE TABLE gamme_maintenance (
      id SERIAL PRIMARY
KEY,
      code VARCHAR(30) NOT
NULL UNIQUE, -- GAM-POM-001
      libelle VARCHAR(200) NOT
NULL,
      type type_maintenance
NOT NULL DEFAULT 'PREVENTIF',
      famille_id INT REFERENCES
famille_equipement(id),
      specialite_id INT REFERENCES
specialite(id),
      duree_estimee_h NUMERIC(6,2)
DEFAULT 1,
      nb_intervenants SMALLINT DEFAULT

```

```

1,
    arret_requis          BOOLEAN NOT NULL
DEFAULT FALSE,
    -- Volet QHSE (differentiant)
    epi_requis            TEXT,          --
casque, gants nitrile, harnais...
    risques_identifies    TEXT,          --
lien avec l'analyse de risque / JSA
    consignation_requise  BOOLEAN NOT NULL
DEFAULT FALSE, -- LOTO
    permis_travail_requis BOOLEAN NOT NULL
DEFAULT FALSE, -- point chaud, espace
confine...
    mode_operatoire_url   TEXT,
    actif                  BOOLEAN NOT NULL
DEFAULT TRUE
);

CREATE TABLE gamme_operation (
    id                      SERIAL PRIMARY KEY,
    gamme_id                INT NOT NULL REFERENCES
gamme_maintenance(id) ON DELETE CASCADE,
    ordre                   SMALLINT NOT NULL,
    libelle                 VARCHAR(200) NOT NULL,
    instruction              TEXT,
    type_controle            VARCHAR(20) DEFAULT
'VISUEL', -- VISUEL, MESURE, REMPLACEMENT,
TEST
    valeur_min              NUMERIC(12,3),
    valeur_max              NUMERIC(12,3),
    unite                   VARCHAR(20),
    obligatoire             BOOLEAN NOT NULL
DEFAULT TRUE,
    UNIQUE (gamme_id, ordre)

```

```
);
```

```
CREATE TABLE gamme_piece (  
    gamme_id          INT NOT NULL REFERENCES  
gamme_maintenance(id) ON DELETE CASCADE,  
    article_id        INT NOT NULL REFERENCES  
article(id),  
    quantite_prevue   NUMERIC(10,2) NOT NULL  
DEFAULT 1,  
    PRIMARY KEY (gamme_id, article_id)  
);
```

```
CREATE TABLE plan_maintenance (  
    id                SERIAL PRIMARY  
KEY,  
    equipement_id     INT NOT NULL  
REFERENCES equipement(id) ON DELETE  
CASCADE,  
    gamme_id          INT NOT NULL  
REFERENCES gamme_maintenance(id),  
    periodicite_type   type_periodicite  
NOT NULL,  
    periodicite_valeur INT NOT NULL CHECK  
(periodicite_valeur > 0),  
    seuil_compteur    NUMERIC(14,2),      -- si declenchement  
par compteur  
    alerte_avant_jours SMALLINT NOT NULL  
DEFAULT 7,  
    date_derniere_execution DATE,  
    date_prochaine_echeance DATE,  
    reglementaire     BOOLEAN NOT NULL  
DEFAULT FALSE, -- controle obligatoire  
(levage, elec, incendie)
```



```

        reference_reglementaire VARCHAR(120),
        actif                     BOOLEAN NOT NULL
DEFAULT TRUE,
        created_at                TIMESTAMPTZ NOT
NULL DEFAULT now(),
        UNIQUE (equipement_id, gamme_id)
);
CREATE INDEX idx_plan_echeance ON
plan_maintenance(date_prochaine_echeance)
WHERE actif;

--
=====
=====
-- 6. FLUX D'INTERVENTION : DI ET ORDRES DE
TRAVAIL
--
=====
=====
CREATE TABLE demande_intervention (
    id                SERIAL PRIMARY KEY,
    numero            VARCHAR(30) NOT NULL
UNIQUE,             -- DI-2026-00001
    equipement_id     INT NOT NULL REFERENCES
equipement(id),
    demandeur_id      INT NOT NULL REFERENCES
utilisateur(id),
    date_demande      TIMESTAMPTZ NOT NULL
DEFAULT now(),
    description        TEXT NOT NULL,
    urgence            priorite_ot NOT NULL
DEFAULT 'P3_NORMALE',
    arret_production   BOOLEAN NOT NULL
DEFAULT FALSE,

```

```

        statut          statut_di NOT NULL
DEFAULT 'NOUVELLE',
        ot_id
INT,                                -- FK
ajoutée plus bas
        motif_rejet      TEXT,
        traite_par        INT REFERENCES
utilisateur(id),
        date_traitement  TIMESTAMPTZ
);

CREATE TABLE cause_defaillance (
        id                SERIAL PRIMARY KEY,
        code              VARCHAR(20) NOT NULL
UNIQUE,
        libelle           VARCHAR(150) NOT NULL,
        categorie         VARCHAR(60) -- 5M :
Matiere, Matériel, Methode, Main d'oeuvre,
Milieu
);

CREATE TABLE ordre_travail (
        id                SERIAL PRIMARY
KEY,
        numero            VARCHAR(30) NOT
NULL UNIQUE, -- OT-2026-00001 (RG-01)
        équipement_id     INT NOT NULL
REFERENCES équipement(id),
        type_maintenance  type_maintenance
NOT NULL,
        origine           origine_ot NOT
NULL DEFAULT 'CREATION_DIRECTE',
        plan_id           INT REFERENCES
plan_maintenance(id),

```

```

    gamme_id INT REFERENCES
gamme_maintenance(id),
    demande_id INT REFERENCES
demande_intervention(id),
    priorite priorite_ot NOT
NULL DEFAULT 'P3_NORMALE',
    statut statut_ot NOT
NULL DEFAULT 'BROUILLON',
    description_demandee TEXT,
    -- Planification
    date_creation TIMESTAMPTZ NOT
NULL DEFAULT now(),
    date_planifiee DATE,
    technicien_responsable_id INT
REFERENCES technicien(id),
    duree_estimee_h NUMERIC(6,2),
    -- Realisation
    date_debut_reelle TIMESTAMPTZ,
    date_fin_reelle TIMESTAMPTZ,
    duree_arret_h NUMERIC(8,2)
DEFAULT 0, -- indisponibilite
equipement
    travaux_realises TEXT,
    diagnostic TEXT,
    cause_id INT REFERENCES
cause_defaillance(id),
    remede TEXT,
    -- Volet QHSE
    permis_travail_requis BOOLEAN NOT NULL
DEFAULT FALSE,
    permis_travail_ref VARCHAR(40),
    consignation_loto BOOLEAN NOT NULL
DEFAULT FALSE,
    analyse_risque_faite BOOLEAN NOT NULL

```

```

DEFAULT FALSE,
    incident_associe      BOOLEAN NOT NULL
DEFAULT FALSE,
    observations_hse      TEXT,
    -- Coûts (RG-07 : calculés par trigger)
    cout_main_oeuvre      NUMERIC(14,2) NOT
NULL DEFAULT 0,
    cout_pieces           NUMERIC(14,2) NOT
NULL DEFAULT 0,
    cout_externe          NUMERIC(14,2) NOT
NULL DEFAULT 0,
    cout_total            NUMERIC(14,2)
GENERATED ALWAYS AS
                        (cout_main_oeuvre
+ cout_pieces + cout_externe) STORED,
    -- Cloture
    valide_par            INT REFERENCES
utilisateur(id),
    date_cloture          TIMESTAMPTZ,
    cree_par              INT REFERENCES
utilisateur(id),
    updated_at            TIMESTAMPTZ NOT
NULL DEFAULT now(),
    deleted_at            TIMESTAMPTZ,
    CONSTRAINT chk_dates_ot CHECK
(date_fin_reelle IS NULL OR
date_debut_reelle IS NULL
                        OR
date_fin_reelle >= date_debut_reelle)
);
ALTER TABLE demande_intervention ADD
CONSTRAINT fk_di_ot FOREIGN KEY (ot_id)
REFERENCES ordre_travail(id);
ALTER TABLE mouvement_stock      ADD

```

```

CONSTRAINT fk_mvt_ot FOREIGN KEY (ot_id)
REFERENCES ordre_travail(id);
CREATE INDEX idx_ot_statut      ON
ordre_travail(statut);
CREATE INDEX idx_ot_equipement ON
ordre_travail(equipement_id);
CREATE INDEX idx_ot_planifiee  ON
ordre_travail(date_planifiee);
CREATE INDEX idx_ot_type_date  ON
ordre_travail(type_maintenance,
date_fin_reelle);

-- Pointage main d'oeuvre (remplace
Interventions_Techniciens + Travaux_Mains
d'Oeuvres)
CREATE TABLE ot_main_oeuvre (
    id          SERIAL PRIMARY KEY,
    ot_id       INT NOT NULL REFERENCES
ordre_travail(id) ON DELETE CASCADE,
    technicien_id INT NOT NULL REFERENCES
technicien(id),
    date_travail DATE NOT NULL,
    heure_debut  TIME NOT NULL,
    heure_fin    TIME NOT NULL,
    duree_h      NUMERIC(6,2) NOT NULL,
    taux_horaire NUMERIC(12,2) NOT NULL
DEFAULT 0,
    cout         NUMERIC(14,2) GENERATED
ALWAYS AS (duree_h * taux_horaire) STORED,
    tache_realisee TEXT,
    created_at   TIMESTAMPTZ NOT NULL
DEFAULT now(),
    CONSTRAINT chk_heures CHECK (heure_fin
> heure_debut)

```

```

);
CREATE INDEX idx_mo_tech_date ON
ot_main_oeuvre(technicien_id,
date_travail);

-- Check-list d'execution (copie de la
gamme a la creation de l'OT)
CREATE TABLE ot_operation (
    id                SERIAL PRIMARY KEY,
    ot_id             INT NOT NULL
REFERENCES ordre_travail(id) ON DELETE
CASCADE,
    gamme_operation_id INT REFERENCES
gamme_operation(id),
    ordre             SMALLINT NOT NULL,
    libelle           VARCHAR(200) NOT
NULL,
    statut            VARCHAR(20) NOT NULL
DEFAULT 'A_FAIRE', -- A_FAIRE, FAIT,
NON_APPLICABLE
    valeur_mesuree    NUMERIC(12,3),
    conforme          BOOLEAN,
    observation       TEXT,
    realise_par       INT REFERENCES
technicien(id),
    realise_le        TIMESTAMPTZ
);

-- Consommation de pieces (remplace
Utilisation pieces, quantite typee)
CREATE TABLE ot_piece (
    id                SERIAL PRIMARY KEY,
    ot_id             INT NOT NULL REFERENCES
ordre_travail(id) ON DELETE CASCADE,

```

```

        article_id      INT NOT NULL REFERENCES
article(id),
        quantite        NUMERIC(12,2) NOT NULL
CHECK (quantite > 0),
        prix_unitaire   NUMERIC(14,2) NOT NULL
DEFAULT 0,
        montant         NUMERIC(16,2) GENERATED
ALWAYS AS (quantite * prix_unitaire)
STORED,
        date_sortie     TIMESTAMPTZ NOT NULL
DEFAULT now(),
        delivre_par     INT REFERENCES
utilisateur(id),
        mouvement_id    BIGINT REFERENCES
mouvement_stock(id)
);

CREATE TABLE piece_jointe (
        id              SERIAL PRIMARY KEY,
        entite          VARCHAR(40) NOT
NULL,          -- EQUIPEMENT, OT, ARTICLE,
GAMME, DI
        entite_id       INT NOT NULL,
        type_document   VARCHAR(40),          -- PHOTO_AVANT,
PHOTO_APRES, FACTURE, MANUEL, PV,
CERTIFICAT
        nom_fichier     VARCHAR(200) NOT NULL,
        url             TEXT NOT NULL,
        taille_ko       INT,
        ajoute_par      INT REFERENCES
utilisateur(id),
        date_ajout      TIMESTAMPTZ NOT NULL
DEFAULT now()

```

```

);
CREATE INDEX idx_pj_entite ON
piece_jointe(entite, entite_id);

--
=====
=====
-- 7. SYSTEME : AUDIT, NOTIFICATIONS,
PARAMETRES
--
=====
=====
CREATE TABLE journal_audit (
    id                BIGSERIAL PRIMARY KEY,
    utilisateur_id    INT REFERENCES
utilisateur(id),
    action            VARCHAR(20) NOT
NULL,               -- CREATE, UPDATE, DELETE,
LOGIN, EXPORT
    table_concernee  VARCHAR(60),
    enregistrement_id VARCHAR(40),
    valeurs_avant    JSONB,
    valeurs_apres    JSONB,
    adresse_ip       INET,
    date_action      TIMESTAMPTZ NOT NULL
DEFAULT now()
);
CREATE INDEX idx_audit_date ON
journal_audit(date_action DESC);

CREATE TABLE notification (
    id                BIGSERIAL PRIMARY KEY,
    destinataire_id  INT NOT NULL REFERENCES
utilisateur(id),

```



```

        type          VARCHAR(40) NOT
NULL,      -- ECHEANCE_PREVENTIF,
STOCK_CRITIQUE, OT_AFFECTE, DI_NOUVELLE
        titre         VARCHAR(150) NOT NULL,
        message       TEXT,
        lien          VARCHAR(200),
        lu            BOOLEAN NOT NULL
DEFAULT FALSE,
        date_creation  TIMESTAMPTZ NOT NULL
DEFAULT now()
);

CREATE TABLE parametre (
        cle            VARCHAR(60) PRIMARY
KEY,
        valeur         TEXT NOT NULL,
        description    TEXT
);
INSERT INTO parametre (cle, valeur,
description) VALUES
('DEWISE','FCFA','Devise de
valorisation'),
('TAUX_HORAIRE_DEFAULT','2500','Cout
horaire par default d'un technicien
(FCFA/h)'),
('JOURS_GENERATION_PREVENTIF','15','Nombre
de jours avant echeance pour generer l'OT
preventif'),
('FORMAT_NUM_OT','OT-{AAAA}-
{00000}','Format de numerotation des OT'),
('HEURES_OUVREES_MOIS','173','Base de
calcul du taux d'occupation');

--

```

```

=====
=====
-- 8. TRIGGERS METIER
--
=====
=====
-- RG-03/RG-04 : toute sortie de piece met
a jour le stock et trace le mouvement
CREATE OR REPLACE FUNCTION
trg_ot_piece_sortie() RETURNS TRIGGER AS $$
DECLARE v_stock NUMERIC(12,2); v_mvt
BIGINT;
BEGIN
    SELECT quantite_stock INTO v_stock FROM
article WHERE id = NEW.article_id FOR
UPDATE;
    IF v_stock < NEW.quantite THEN
        RAISE EXCEPTION 'Stock insuffisant
pour l'article % (dispo: %, demande: %)',
NEW.article_id, v_stock,
NEW.quantite;
    END IF;
    UPDATE article SET quantite_stock =
quantite_stock - NEW.quantite WHERE id =
NEW.article_id;
    INSERT INTO mouvement_stock
(article_id, type_mvt, quantite,
prix_unitaire, ot_id,
motif,
stock_avant, stock_apres, utilisateur_id)
VALUES
(NEW.article_id, 'SORTIE', NEW.quantite, NEW.p
rix_unitaire, NEW.ot_id,
'Sortie sur OT', v_stock,

```

```

v_stock - NEW.quantite, NEW.delivre_par)
    RETURNING id INTO v_mvt;
    NEW.mouvement_id := v_mvt;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER tg_ot_piece_sortie BEFORE
INSERT ON ot_piece
FOR EACH ROW EXECUTE FUNCTION
trg_ot_piece_sortie();

-- RG-07 : recalcul automatique des couts
de l'OT
CREATE OR REPLACE FUNCTION
trg_maj_cout_ot() RETURNS TRIGGER AS $$
DECLARE v_ot INT;
BEGIN
    v_ot := COALESCE(NEW.ot_id, OLD.ot_id);
    UPDATE ordre_travail o SET
        cout_main_oeuvre = COALESCE((SELECT
SUM(cout) FROM ot_main_oeuvre WHERE ot_id =
v_ot),0),
        cout_pieces      = COALESCE((SELECT
SUM(montant) FROM ot_piece      WHERE ot_id
= v_ot),0),
        updated_at       = now()
    WHERE o.id = v_ot;
    RETURN NULL;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER tg_cout_mo    AFTER INSERT
OR UPDATE OR DELETE ON ot_main_oeuvre
FOR EACH ROW EXECUTE FUNCTION
trg_maj_cout_ot();

```

```

CREATE TRIGGER tg_cout_piece AFTER INSERT
OR UPDATE OR DELETE ON ot_piece
FOR EACH ROW EXECUTE FUNCTION
trg_maj_cout_ot();

-- RG-09 : a la cloture d'un OT preventif,
recalcul de la prochaine echeance
CREATE OR REPLACE FUNCTION trg_cloture_ot()
RETURNS TRIGGER AS $$
DECLARE p RECORD;
BEGIN
    IF NEW.statut = 'CLOTURE' AND
    OLD.statut <> 'CLOTURE' AND NEW.plan_id IS
    NOT NULL THEN
        SELECT * INTO p FROM
plan_maintenance WHERE id = NEW.plan_id;
        UPDATE plan_maintenance SET
            date_derniere_execution =
COALESCE(NEW.date_fin_reelle::date,
CURRENT_DATE),
            date_prochaine_echeance = CASE
p.periodicite_type
                WHEN 'JOUR' THEN
COALESCE(NEW.date_fin_reelle::date, CURRENT_
DATE) + (p.periodicite_valeur || '
days')::interval
                WHEN 'SEMAINE' THEN
COALESCE(NEW.date_fin_reelle::date, CURRENT_
DATE) + (p.periodicite_valeur || '
weeks')::interval
                WHEN 'MOIS' THEN
COALESCE(NEW.date_fin_reelle::date, CURRENT_
DATE) + (p.periodicite_valeur || '
months')::interval

```

```

        WHEN 'ANNEE' THEN
COALESCE(NEW.date_fin_reelle::date,CURRENT_
DATE) + (p.periodicite_valeur || '
years')::interval
        ELSE
p.date_prochaine_echeance END
        WHERE id = NEW.plan_id;
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER tg_cloture_ot AFTER UPDATE
ON ordre_travail
FOR EACH ROW EXECUTE FUNCTION
trg_cloture_ot();

--
=====
=====
-- 9. VUES DE PILOTAGE (KPI)
--
=====
=====
-- Reprise et fiabilisation des requetes
Access existantes
CREATE VIEW v_stock_critique AS
SELECT a.ref_article, a.designation,
a.quantite_stock, a.seuil_reappro,
(a.seuil_reappro - a.quantite_stock)
AS quantite_a_commander,
f.raison_sociale AS fournisseur,
a.delai_appro_jours
FROM article a LEFT JOIN fournisseur f ON
f.id = a.fournisseur_principal_id

```

```
WHERE a.actif AND a.quantite_stock <=
a.seuil_reappro;
```

```
CREATE VIEW v_valeur_stock AS
SELECT COUNT(*) AS nb_references,
       SUM(quantite_stock) AS
quantite_totale,
       SUM(quantite_stock *
prix_unitaire_moyen) AS valeur_fcfa
FROM article WHERE actif;
```

```
CREATE VIEW v_echeancier_preventif AS
SELECT e.code_equipement, e.designation,
l.libelle AS localisation,
       g.libelle AS gamme,
p.date_prochaine_echeance,
       (p.date_prochaine_echeance -
CURRENT_DATE) AS jours_restants,
       p.reglementaire
FROM plan_maintenance p
JOIN equipement e ON e.id = p.equipement_id
JOIN gamme_maintenance g ON g.id =
p.gamme_id
LEFT JOIN localisation l ON l.id =
e.localisation_id
WHERE p.actif ORDER BY
p.date_prochaine_echeance;
```

```
CREATE VIEW v_charge_technicien AS
SELECT t.matricule, t.nom_prenom, s.libelle
AS specialite,
       date_trunc('month', m.date_travail)
AS mois,
       COUNT(DISTINCT m.ot_id) AS nb_ot,
```

```

SUM(m.duree_h)          AS
heures_travaillees,
SUM(m.cout)             AS
cout_mo_fcfa
FROM technicien t
LEFT JOIN ot_main_oeuvre m ON
m.technicien_id = t.id
LEFT JOIN specialite s ON s.id =
t.specialite_id
GROUP BY t.matricule, t.nom_prenom,
s.libelle, date_trunc('month',
m.date_travail);

-- MTBF / MTTR / disponibilite par
equipement (base : 12 derniers mois)
CREATE VIEW v_indicateurs_equipement AS
WITH pannes AS (
    SELECT equipement_id,
           COUNT(*) AS nb_defaillances,
           SUM(duree_arret_h) AS
heures_arret,
           SUM(EXTRACT(EPOCH FROM
(date_fin_reelle -
date_debut_reelle))/3600) AS
heures_reparation
    FROM ordre_travail
    WHERE type_maintenance = 'CORRECTIF' AND
statut = 'CLOTURE'
    AND date_fin_reelle >= now() - interval
'12 months'
    GROUP BY equipement_id
)
SELECT e.code_equipement, e.designation,
e.criticite,

```

```

COALESCE(p.nb_defaillances,0)
        AS nb_defaillances,

COALESCE(p.heures_arret,0)
        AS heures_arret,
        CASE WHEN
COALESCE(p.nb_defaillances,0) > 0
        THEN (8760 -
COALESCE(p.heures_arret,0)) /
p.nb_defaillances END AS mtbf_h,
        CASE WHEN
COALESCE(p.nb_defaillances,0) > 0
        THEN p.heures_reparation /
p.nb_defaillances END AS mttr_h,
        ROUND(((8760 -
COALESCE(p.heures_arret,0)) / 8760 *
100)::numeric, 2) AS
taux_disponibilite_pct,
        (SELECT SUM(cout_total) FROM
ordre_travail o
        WHERE o.equipement_id = e.id AND
o.statut='CLOTURE'
        AND o.date_fin_reelle >= now() -
interval '12 months') AS
cout_maintenance_12m
FROM equipment e LEFT JOIN pannes p ON
p.equipement_id = e.id
WHERE e.deleted_at IS NULL;

-- Ratio preventif / correctif (cible : 70
/ 30)
CREATE VIEW v_ratio_preventif AS
SELECT date_trunc('month', date_fin_reelle)

```



```

AS mois,
    COUNT(*) FILTER (WHERE
type_maintenance IN
('PREVENTIF','REGLEMENTAIRE')) AS
nb_preventif,
    COUNT(*) FILTER (WHERE
type_maintenance =
'CORRECTIF') AS
nb_correctif,
    ROUND(100.0 * COUNT(*) FILTER (WHERE
type_maintenance IN
('PREVENTIF','REGLEMENTAIRE'))
/ NULLIF(COUNT(*),0),
1)
AS taux_preventif_pct
FROM ordre_travail WHERE statut = 'CLOTURE'
AND date_fin_reelle IS NOT NULL
GROUP BY 1 ORDER BY 1 DESC;

--
=====
=====
-- 10. JEU DE DONNEES INITIAL (extrait de
la base Access)
--
=====
=====
INSERT INTO specialite (code, libelle)
VALUES
('MEC','Mecanicien'),
('ELEC','Electricien'), ('SOUD','Soudeur'),
('HYD','Hydraulicien'),
('INSTR','Instrumentiste'),
('POLY','Polyvalent');

```

```
INSERT INTO technicien (matricule,
nom_prenom, specialite_id, statut) VALUES
('S010','SOGOBA Issa',
(SELECT id FROM specialite WHERE
code='MEC'), 'ACTIF'),
('S011','KOUAKOU Kouame Rubin',
(SELECT id FROM specialite WHERE
code='MEC'), 'ACTIF'),
('S012','BAMBA Bakary Siriky',
(SELECT id FROM specialite WHERE
code='MEC'), 'ACTIF'),
('S014','DIARASSOUBA Ben Swaliho',
(SELECT id FROM specialite WHERE
code='MEC'), 'ACTIF'),
('S015','KOUAME Koffi JB',
(SELECT id FROM specialite WHERE
code='MEC'), 'ACTIF'),
('S016','SEKONGO Kiboni',
(SELECT id FROM specialite WHERE
code='MEC'), 'ACTIF'),
('S017','DOUMBIA Idrissa',
(SELECT id FROM specialite WHERE
code='MEC'), 'ACTIF'),
('S019','KOFFI Renaud',
(SELECT id FROM specialite WHERE
code='SOUD'),'ACTIF'),
('S022','BOHOUSSOU N''dri Germain',
(SELECT id FROM specialite WHERE
code='SOUD'),'ACTIF'),
('S028','ZORO Bi Arnaud',
(SELECT id FROM specialite WHERE
code='ELEC'),'ACTIF'),
('S036','KONAN Kouame Olivier',
```

```

(SELECT id FROM specialite WHERE
code='SOUD'),'ACTIF');

INSERT INTO role (code, libelle) VALUES
('ADMIN','Administrateur'),
('RESP_MAINT','Responsable maintenance'),
('PLANIF','Planificateur'),
('TECH','Technicien'),
('MAGASIN','Magasinier'),
('DEMANDEUR','Demandeur / exploitation'),
('DIRECTION','Direction (lecture)'),
('QHSE','Responsable QHSE');

INSERT INTO cause_defaillance (code,
libelle, categorie) VALUES
('USU','Usure normale','Materiel'),
('LUB','Defaut de
lubrification','Methode'),
('SUR','Surcharge /
surintensite','Milieu'), ('HUM','Erreur
humaine / mauvaise utilisation','Main
d'oeuvre'),
('PIE','Piece defectueuse','Matiere'),
('COR','Corrosion','Milieu'),
('MTG','Defaut de montage','Methode'),
('ENV','Conditions
environnementales','Milieu'),
('INC','Cause inconnue','Autre');
--
=====
=====
-- FIN DU SCRIPT
--
=====

```

```

=====

--

=====
=====
-- EXTENSION V1.1
-- Fonctionnement hors ligne (mobile /
APK) et extensibilite du parc
--
=====
=====

-- -----
-----
-- 11. EXTENSIBILITE DU PARC
-- -----
-----
CREATE TYPE statut_fiche AS ENUM
('A_VALIDER', 'VALIDEE', 'REJETEE');

ALTER TABLE equipement
  ADD COLUMN statut_fiche
statut_fiche NOT NULL DEFAULT 'VALIDEE', -
- RG-27
  ADD COLUMN cree_depuis_mobile
BOOLEAN      NOT NULL DEFAULT FALSE,
  ADD COLUMN valide_par      INT
REFERENCES utilisateur(id),
  ADD COLUMN date_validation
TIMESTAMPZ;

-- Un equipement au statut A_VALIDER est
utilisable pour rattacher une
-- intervention mais exclu des indicateurs

```

```

et de l'inventaire.
CREATE INDEX idx_equip_statut_fiche ON
equipement(statut_fiche)
  WHERE statut_fiche = 'A_VALIDER';

-- Champs techniques definis par
l'administrateur, sans redevelopement.
-- Les valeurs sont stockees dans
equipement.caracteristiques (JSONB),
-- indexees par le code du champ.
CREATE TABLE champ_personnalise (
  id              SERIAL PRIMARY KEY,
  famille_id      INT REFERENCES
famille_equipement(id) ON DELETE CASCADE,
  code            VARCHAR(40) NOT
NULL,            -- cle utilisee dans le
JSONB
  libelle         VARCHAR(120) NOT NULL,
  type_champ      VARCHAR(20) NOT
NULL,            -- TEXTE, NOMBRE, DATE,
LISTE, BOOLEEN
  unite          VARCHAR(20),
  valeurs_possibles
JSONB,            -- pour le
type LISTE
  obligatoire     BOOLEAN NOT NULL
DEFAULT FALSE,
  ordre_affichage SMALLINT NOT NULL
DEFAULT 1,
  actif           BOOLEAN NOT NULL
DEFAULT TRUE,
  UNIQUE (famille_id, code)
);
-- Recherche sur les caracteristiques

```

```

variables
CREATE INDEX idx_equip_caracteristiques ON
equipement USING gin (caracteristiques);

-- Tracabilite des imports Excel en masse
(annulation possible)
CREATE TABLE import_lot (
    id SERIAL PRIMARY KEY,
    type_import VARCHAR(30) NOT
NULL, -- EQUIPEMENT, ARTICLE,
TECHNICIEN
    nom_fichier VARCHAR(200),
    nb_lignes INT, nb_succes INT,
nb_erreurs INT,
    rapport
JSONB, -- erreurs
    ligne a ligne
    annule BOOLEAN NOT NULL
DEFAULT FALSE,
    importe_par INT REFERENCES
utilisateur(id),
    date_import TIMESTAMPTZ NOT NULL
DEFAULT now()
);
ALTER TABLE equipement ADD COLUMN
import_lot_id INT REFERENCES
import_lot(id);
ALTER TABLE article ADD COLUMN
import_lot_id INT REFERENCES
import_lot(id);

-- -----
-----
-- 12. TERMINAUX ET SYNCHRONISATION HORS

```

LIGNE

```
-- -----  
-----  
CREATE TABLE terminal (  
    id                SERIAL PRIMARY KEY,  
    identifiant       VARCHAR(80) NOT NULL  
UNIQUE, -- genere a l'installation  
    utilisateur_id    INT REFERENCES  
utilisateur(id),  
    modele            VARCHAR(80),  
    version_os        VARCHAR(40),  
    version_app       VARCHAR(20),  
    derniere_sync     TIMESTAMPTZ,  
    elements_en_attente INT DEFAULT 0,  
    revoque            BOOLEAN NOT NULL  
DEFAULT FALSE, -- RG-29 (perte / vol)  
    date_revocation   TIMESTAMPTZ,  
    created_at        TIMESTAMPTZ NOT NULL  
DEFAULT now()  
);  
  
CREATE TABLE sync_lot (  
    id                BIGSERIAL PRIMARY KEY,  
    terminal_id       INT REFERENCES  
terminal(id),  
    utilisateur_id    INT REFERENCES  
utilisateur(id),  
    date_reception    TIMESTAMPTZ NOT NULL  
DEFAULT now(),  
    nb_operations     INT NOT NULL DEFAULT 0,  
    nb_acceptees      INT NOT NULL DEFAULT 0,  
    nb_doublons       INT NOT NULL DEFAULT  
0, -- ignores par idempotence (RG-23)  
    nb_conflits       INT NOT NULL DEFAULT 0,
```

```

        nb_rejetees      INT NOT NULL DEFAULT 0,
        version_app      VARCHAR(20),
        detail
JSONB                      -- motifs de
rejet, arbitrages
);
CREATE INDEX idx_sync_date ON
sync_lot(date_reception DESC);

-- Identifiant genere par le terminal :
garantit l'idempotence (RG-23).
-- Une meme operation renvoyee apres
coupure ne cree qu'un enregistrement.
ALTER TABLE ordre_travail      ADD COLUMN
client_uuid UUID UNIQUE;
ALTER TABLE demande_intervention ADD COLUMN
client_uuid UUID UNIQUE;
ALTER TABLE ot_main_oeuvre      ADD COLUMN
client_uuid UUID UNIQUE;
ALTER TABLE ot_piece            ADD COLUMN
client_uuid UUID UNIQUE;
ALTER TABLE ot_operation        ADD COLUMN
client_uuid UUID UNIQUE;
ALTER TABLE piece_jointe        ADD COLUMN
client_uuid UUID UNIQUE;
ALTER TABLE compteur_releve      ADD COLUMN
client_uuid UUID UNIQUE;
ALTER TABLE equipement          ADD COLUMN
client_uuid UUID UNIQUE;

-- Horodatage : saisie terrain (RG-24)
distinct de la reception serveur
ALTER TABLE ordre_travail      ADD COLUMN
saisi_hors_ligne BOOLEAN NOT NULL DEFAULT

```



```

FALSE,
                                ADD COLUMN
date_saisie_terrain TIMESTAMPTZ,
                                ADD COLUMN
date_synchronisation TIMESTAMPTZ;
ALTER TABLE ot_main_oeuvre ADD COLUMN
date_saisie_terrain TIMESTAMPTZ,
                                ADD COLUMN
date_synchronisation TIMESTAMPTZ;
ALTER TABLE piece_jointe ADD COLUMN
date_prise_vue TIMESTAMPTZ,
                                ADD COLUMN
latitude NUMERIC(10,7),
                                ADD COLUMN
longitude NUMERIC(10,7),
                                ADD COLUMN
synchronisee BOOLEAN NOT NULL DEFAULT TRUE;

-- Demande de sortie de piece saisie hors
-- ligne : ne devient un mouvement
-- de stock qu'apres validation du
-- magasinier (RG-26).
CREATE TYPE statut_demande_piece AS ENUM
('EN_ATTENTE', 'VALIDEE', 'REFUSEE');

CREATE TABLE demande_piece (
    id SERIAL PRIMARY KEY,
    client_uuid UUID UNIQUE,
    ot_id INT NOT NULL REFERENCES
ordre_travail(id) ON DELETE CASCADE,
    article_id INT NOT NULL REFERENCES
article(id),
    quantite NUMERIC(12,2) NOT NULL
CHECK (quantite > 0),

```

```

        demande_par      INT REFERENCES
technicien(id),
        date_demande     TIMESTAMPTZ NOT NULL
DEFAULT now(),
        statut           statut_demande_piece
NOT NULL DEFAULT 'EN_ATTENTE',
        traite_par       INT REFERENCES
utilisateur(id),
        date_traitement  TIMESTAMPTZ,
        motif_refus      TEXT,
        ot_piece_id      INT REFERENCES
ot_piece(id) -- renseigne apres
validation
);
CREATE INDEX idx_demande_piece_statut ON
demande_piece(statut) WHERE statut =
'EN_ATTENTE';

-- Parametres complementaires
INSERT INTO parametre (cle, valeur,
description) VALUES

('DUREE_SESSION_HORS_LIGNE_JOURS','7','Vali
dite d''une session hors ligne avant
reconnexion obligatoire (RG-28)'),
('QUOTA_LOCAL_MO','500','Empreinte
maximale des donnees et photos sur le
terminal'),

('PHOTO_LARGEUR_MAX_PX','1600','Redimension
nement des photos avant stockage local'),
('PHOTO_QUALITE','75','Qualite de
compression des photos en pourcentage'),
('VERSION_APP_MIN','1.0.0','Version

```

```

minimale de l''application mobile acceptee
par le serveur'),
('JOURS_OT_PRECHARGES','7','Profondeur de
prechargement des ordres de travail sur le
terminal');

-- Vue de supervision de la flotte de
terminaux
CREATE VIEW v_etat_terminaux AS
SELECT t.identifiant, u.nom, u.prenom,
t.modele, t.version_app,
t.derniere_sync,
(now() - t.derniere_sync) AS
anciennete_sync,
t.elements_en_attente, t.revoque
FROM terminal t LEFT JOIN utilisateur u ON
u.id = t.utilisateur_id
ORDER BY t.derniere_sync NULLS FIRST;

-- Equipements crees sur le terrain et en
attente de validation (RG-27)
CREATE VIEW v_equipements_a_valider AS
SELECT e.id, e.code_equipement,
e.designation, l.libelle AS localisation,
e.created_at, u.nom AS cree_par
FROM equipement e
LEFT JOIN localisation l ON l.id =
e.localisation_id
LEFT JOIN utilisateur u ON u.id =
e.valide_par
WHERE e.statut_fiche = 'A_VALIDER' AND
e.deleted_at IS NULL;
--
=====

```

```

=====
-- FIN DE L'EXTENSION V1.1
--
=====
=====

--
=====
=====
-- EXTENSION V2.0
-- Domaines Production, Produit fini /
Expeditions, Laboratoire
-- Moteur generique de validation et de
signature
--
=====
=====

-- -----
-----
-- 13. MOTEUR GENERIQUE DE VALIDATION ET DE
SIGNATURE
--      Developpe une seule fois, parametre
pour chaque type de document.
-- -----
-----
CREATE TYPE statut_validation AS ENUM

('BROUILLON','SOUMIS','VERIFIE','APPROUVE',
'DIFFUSE','RETOURNE','ANNULE');
CREATE TYPE action_validation AS ENUM
('VERIFIER','APPROUVER','SIGNER','DIFFUSER'
);

```

```

CREATE TABLE circuit_validation (
    id                SERIAL PRIMARY KEY,
    code              VARCHAR(40) NOT NULL
    UNIQUE,    -- RAPPORT_QUART,
    BULLETIN_ANALYSE...
    entite            VARCHAR(40) NOT
    NULL,            -- table concernee
    libelle            VARCHAR(120) NOT NULL,
    delai_relance_h    SMALLINT DEFAULT
    24,              -- RG-55
    actif              BOOLEAN NOT NULL
    DEFAULT TRUE
);

CREATE TABLE circuit_etape (
    id                SERIAL PRIMARY KEY,
    circuit_id         INT NOT NULL REFERENCES
    circuit_validation(id) ON DELETE CASCADE,
    ordre              SMALLINT NOT NULL,
    role_id            INT NOT NULL REFERENCES
    role(id),
    action              action_validation NOT
    NULL,
    obligatoire        BOOLEAN NOT NULL
    DEFAULT TRUE,
    UNIQUE (circuit_id, ordre)
);

CREATE TABLE validation (
    id                BIGSERIAL PRIMARY KEY,
    entite              VARCHAR(40) NOT NULL,
    entite_id           INT NOT NULL,
    etape_id            INT REFERENCES
    circuit_etape(id),

```

```

        utilisateur_id INT NOT NULL REFERENCES
utilisateur(id),
        decision        VARCHAR(20) NOT
NULL,                -- APPROUVE, RETOURNE,
REJETE
        commentaire
TEXT,                --
obligatoire si RETOURNE (RG-51)
        date_decision  TIMESTAMPTZ NOT NULL
DEFAULT now(),
        empreinte
VARCHAR(64),        --
identifiant de controle du document
        adresse_ip      INET
);
CREATE INDEX idx_validation_entite ON
validation(entite, entite_id);

-- Separation des taches (RG-50) : un meme
utilisateur ne peut occuper
-- deux etapes consecutives d'un meme
circuit.
CREATE OR REPLACE FUNCTION
trg_separation_taches() RETURNS TRIGGER AS
$$
DECLARE v_precedent INT;
BEGIN
        SELECT utilisateur_id INTO v_precedent
        FROM validation
        WHERE entite = NEW.entite AND
entite_id = NEW.entite_id
        AND decision = 'APPROUVE'
        ORDER BY date_decision DESC LIMIT 1;
        IF v_precedent IS NOT NULL AND

```

```

v_precedent = NEW.utilisateur_id
    AND NEW.decision = 'APPROUVE' THEN
        RAISE EXCEPTION 'Separation des
taches : cet utilisateur a deja valide
l''etape precedente de ce document';
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER tg_separation_taches BEFORE
INSERT ON validation
FOR EACH ROW EXECUTE FUNCTION
trg_separation_taches();

-- -----
-- -----
-- 14. DOMAINE PRODUCTION
-- -----
-----

CREATE TYPE type_produit AS ENUM

('MATIERE_PREMIERE','PRODUIT_FINI','SOUS_PR
ODUIT','CONSOMMABLE');
CREATE TYPE type_mvt_produit AS ENUM

('RECEPTION','SORTIE_PRODUCTION','PRODUCTIO
N','EXPEDITION','AJUSTEMENT','INVENTAIRE');
CREATE TYPE statut_demande_matiere AS ENUM
('DEMANDEE','SERVIE','PARTIELLE','REFUSEE',
'ANNULEE');

-- Produits, sous-produits : entierement
parametrable (RG-36).
-- Ajouter un quatrieme sous-produit = une

```

ligne, pas une livraison.

```
CREATE TABLE produit (  
    id                SERIAL PRIMARY KEY,  
    code              VARCHAR(30) NOT  
NULL UNIQUE,  
    libelle           VARCHAR(150) NOT  
NULL,  
    type              type_produit NOT  
NULL,  
    unite             VARCHAR(20) NOT  
NULL DEFAULT 'kg',  
    densite_reference NUMERIC(8,4),      -- conversion  
    litres <-> kg  
    quantite_stock    NUMERIC(14,2) NOT  
NULL DEFAULT 0,  
    seuil_alerte      NUMERIC(14,2),  
    actif             BOOLEAN NOT NULL  
DEFAULT TRUE,  
    created_at        TIMESTAMPTZ NOT  
NULL DEFAULT now()  
);
```

```
CREATE TABLE ligne_production (  
    id                SERIAL PRIMARY KEY,  
    code              VARCHAR(20) NOT NULL  
UNIQUE,  
    libelle           VARCHAR(120) NOT NULL,  
    site_id           INT REFERENCES  
site(id),  
    capacite_t_jour   NUMERIC(10,2),  
    actif             BOOLEAN NOT NULL  
DEFAULT TRUE  
);
```



```

CREATE TABLE demande_matiere (
    id                SERIAL PRIMARY KEY,
    numero            VARCHAR(30) NOT
NULL UNIQUE,      -- DM-AAAA-NNNNN
    client_uuid       UUID UNIQUE,
    date_demande      TIMESTAMPTZ NOT
NULL DEFAULT now(),
    produit_id        INT NOT NULL
REFERENCES produit(id),
    ligne_id          INT REFERENCES
ligne_production(id),
    quart
CHAR(1),           -- A, B, C
    quantite_demandee NUMERIC(14,2) NOT
NULL CHECK (quantite_demandee > 0),
    quantite_servie   NUMERIC(14,2),
    motif_ecart
TEXT,              --
obligatoire si ecart (RG-31)
    demandeur_id      INT REFERENCES
utilisateur(id),
    magasinier_id     INT REFERENCES
utilisateur(id),
    date_service      TIMESTAMPTZ,
    statut
statut_demande_matiere NOT NULL DEFAULT
'DEMANDEE',
    journal_quart_id  INT
);

CREATE TABLE journal_quart (
    id                SERIAL PRIMARY KEY,
    numero            VARCHAR(30) NOT

```

```

NULL UNIQUE,    -- JQ-AAAA-NNNNN
    client_uuid        UUID UNIQUE,
    date_journee        DATE NOT NULL,
    quart              CHAR(1) NOT
NULL,           -- A, B, C
    ligne_id           INT NOT NULL
REFERENCES ligne_production(id),
    chef_quart_id       INT NOT NULL
REFERENCES utilisateur(id),
    heure_debut         TIMESTAMPTZ,
    heure_fin           TIMESTAMPTZ,
    heures_fonctionnement NUMERIC(6,2),
    -- Totaux calculés par trigger
    total_entrees_kg     NUMERIC(14,2) NOT
NULL DEFAULT 0,
    total_sorties_kg     NUMERIC(14,2) NOT
NULL DEFAULT 0,
    ecart_kg            NUMERIC(14,2) NOT
NULL DEFAULT 0,
    ecart_pct           NUMERIC(6,3),
    rendement_pct       NUMERIC(6,3),
    observations        TEXT,
    commentaire_ecart
TEXT,           --
obligatoire au-dela du seuil
    statut              statut_validation
NOT NULL DEFAULT 'BROUILLON',
    rapport_rectifie_id INT REFERENCES
journal_quart(id),    -- RG-34
    created_at          TIMESTAMPTZ NOT
NULL DEFAULT now(),
    updated_at          TIMESTAMPTZ NOT
NULL DEFAULT now(),
    UNIQUE (date_journee, quart, ligne_id)

```

```

);
ALTER TABLE demande_matiere
    ADD CONSTRAINT fk_dm_journal FOREIGN KEY
(journal_quart_id) REFERENCES
journal_quart(id);
CREATE INDEX idx_jq_date ON
journal_quart(date_journee DESC);
CREATE INDEX idx_jq_statut ON
journal_quart(statut);

CREATE TABLE journal_entree (
    id SERIAL PRIMARY KEY,
    journal_quart_id INT NOT NULL
REFERENCES journal_quart(id) ON DELETE
CASCADE,
    produit_id INT NOT NULL
REFERENCES produit(id),
    quantite_kg NUMERIC(14,2) NOT
NULL CHECK (quantite_kg > 0),
    lot_matiere
VARCHAR(40),
    --
    tracabilite_amont
demande_matiere_id INT REFERENCES
demande_matiere(id),
    observation TEXT
);

CREATE TABLE journal_sortie (
    id SERIAL PRIMARY KEY,
    journal_quart_id INT NOT NULL
REFERENCES journal_quart(id) ON DELETE
CASCADE,
    produit_id INT NOT NULL
REFERENCES produit(id),

```

```

        quantite_kg          NUMERIC(14,2) NOT
NULL CHECK (quantite_kg >= 0),
        tank_id
INT,                                -- si
produit fini
        destination
VARCHAR(120),                        -- magasin
sous-produits, vrac...
        observation          TEXT
);

CREATE TABLE journal_arret (
        id                  SERIAL PRIMARY KEY,
        journal_quart_id    INT NOT NULL
REFERENCES journal_quart(id) ON DELETE
CASCADE,
        equipement_id       INT REFERENCES
equipement(id),
        type_arret          VARCHAR(40) NOT
NULL,                                -- PANNE, REGLAGE, ENERGIE,
MP, NETTOYAGE
        cause               TEXT,
        heure_debut         TIMESTAMPTZ,
        duree_min           INT NOT NULL CHECK
(duree_min > 0),
        demande_intervention_id INT REFERENCES
demande_intervention(id) -- RG-35
);

CREATE TABLE mouvement_produit (
        id                  BIGSERIAL PRIMARY KEY,
        produit_id          INT NOT NULL REFERENCES
produit(id),
        type_mvt            type_mvt_produit NOT

```

```

NULL,
    quantite_kg      NUMERIC(14,2) NOT NULL,
    date_mvt        TIMESTAMPTZ NOT NULL
DEFAULT now(),
    journal_quart_id INT REFERENCES
journal_quart(id),
    demande_matiere_id INT REFERENCES
demande_matiere(id),
    stock_avant      NUMERIC(14,2),
    stock_apres      NUMERIC(14,2),
    utilisateur_id   INT REFERENCES
utilisateur(id),
    motif            TEXT
);
CREATE INDEX idx_mvt_produit ON
mouvement_produit(produit_id, date_mvt
DESC);

-- Bilan matiere et rendement recalculés à
chaque modification (RG-30)
CREATE OR REPLACE FUNCTION
trg_bilan_matiere() RETURNS TRIGGER AS $$
DECLARE v_jq INT; v_ent NUMERIC; v_sor
NUMERIC; v_pf NUMERIC;
BEGIN
    v_jq := COALESCE(NEW.journal_quart_id,
OLD.journal_quart_id);
    SELECT COALESCE(SUM(quantite_kg),0)
INTO v_ent FROM journal_entree WHERE
journal_quart_id = v_jq;
    SELECT COALESCE(SUM(quantite_kg),0)
INTO v_sor FROM journal_sortie WHERE
journal_quart_id = v_jq;
    SELECT COALESCE(SUM(s.quantite_kg),0)

```

```

    INTO v_pf
        FROM journal_sortie s JOIN produit p
    ON p.id = s.produit_id
        WHERE s.journal_quart_id = v_jq AND
    p.type = 'PRODUIT_FINI';
    UPDATE journal_quart SET
        total_entrees_kg = v_ent,
        total_sorties_kg = v_sor,
        ecart_kg          = v_ent - v_sor,
        ecart_pct          = CASE WHEN v_ent >
0 THEN ROUND(((v_ent -
v_sor)/v_ent*100)::numeric,3) END,
        rendement_pct    = CASE WHEN v_ent >
0 THEN ROUND((v_pf/v_ent*100)::numeric,3)
    END,
        updated_at       = now()
    WHERE id = v_jq;
    RETURN NULL;
END; $$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER tg_bilan_entree AFTER INSERT
OR UPDATE OR DELETE ON journal_entree
FOR EACH ROW EXECUTE FUNCTION
    trg_bilan_matiere();
CREATE TRIGGER tg_bilan_sortie AFTER INSERT
OR UPDATE OR DELETE ON journal_sortie
FOR EACH ROW EXECUTE FUNCTION
    trg_bilan_matiere();

```

```

-- -----
-----
-- 15. DOMAINE PRODUIT FINI, STOCKAGE ET
EXPEDITIONS
-- -----

```

```

-----
CREATE TYPE type_mvt_tank AS ENUM

('ENTREE_PRODUCTION','TRANSFERT_ENTREE','TR
ANSFERT_SORTIE','CHARGEMENT','AJUSTEMENT_JA
UGE');
CREATE TYPE type_expedition AS ENUM
('CONTENEUR_FLEXITANK','CAMION_CITERNE','FU
T','VRAC');
CREATE TYPE statut_commande_client AS ENUM
('OUVERTE','PARTIELLE','LIVREE','ANNULEE');

CREATE TABLE tank (
    id                SERIAL PRIMARY KEY,
    code              VARCHAR(20) NOT
NULL UNIQUE,      -- TK-01
    libelle           VARCHAR(120),
    produit_id        INT REFERENCES
produit(id),
    site_id           INT REFERENCES
site(id),
    capacite_litres   NUMERIC(14,2) NOT
NULL,
    stock_litres      NUMERIC(14,2) NOT
NULL DEFAULT 0 CHECK (stock_litres >= 0),
    stock_kg          NUMERIC(14,2) NOT
NULL DEFAULT 0,
    litres_reserves   NUMERIC(14,2) NOT
NULL DEFAULT 0,
    seuil_haut_pct    SMALLINT DEFAULT
90,
    seuil_bas_pct     SMALLINT DEFAULT
10,
    bareme_jaugeage

```

```

JSONB,                                -- table
hauteur (cm) -> volume (l)
    statut                            VARCHAR(20) NOT
NULL DEFAULT 'EN_SERVICE',
    actif                            BOOLEAN NOT NULL
DEFAULT TRUE
);
ALTER TABLE journal_sortie ADD CONSTRAINT
fk_js_tank FOREIGN KEY (tank_id) REFERENCES
tank(id);

CREATE TABLE tank_mouvement (
    id                                BIGSERIAL PRIMARY
KEY,
    tank_id                          INT NOT NULL
REFERENCES tank(id),
    type_mvt                          type_mvt_tank NOT
NULL,
    quantite_litres                   NUMERIC(14,2) NOT
NULL,
    quantite_kg                       NUMERIC(14,2),
    temperature                       NUMERIC(6,2),
    densite                           NUMERIC(8,4),
    date_mvt                          TIMESTAMPTZ NOT
NULL DEFAULT now(),
    journal_quart_id                  INT REFERENCES
journal_quart(id),
    chargement_id                     INT,
    tank_destination_id               INT REFERENCES
tank(id),
    stock_avant_litres                NUMERIC(14,2),
    stock_apres_litres                NUMERIC(14,2),
    motif                             TEXT,
    utilisateur_id                    INT REFERENCES

```



```

utilisateur(id)
);
CREATE INDEX idx_tank_mvt ON
tank_mouvement(tank_id, date_mvt DESC);

CREATE TABLE jaugeage (
    id                SERIAL PRIMARY KEY,
    tank_id           INT NOT NULL
REFERENCES tank(id),
    date_jaugeage     TIMESTAMPTZ NOT
NULL DEFAULT now(),
    hauteur_cm        NUMERIC(8,2),
    volume_litres     NUMERIC(14,2) NOT
NULL,
    temperature       NUMERIC(6,2),
    densite           NUMERIC(8,4),
    masse_kg          NUMERIC(14,2),
    stock_theorique_l NUMERIC(14,2),
    ecart_litres      NUMERIC(14,2),
    ecart_pct         NUMERIC(6,3),
    effectue_par      INT REFERENCES
utilisateur(id),
    observation       TEXT
);

CREATE TABLE client (
    id                SERIAL PRIMARY KEY,
    code              VARCHAR(20) NOT NULL
UNIQUE,
    raison_sociale    VARCHAR(150) NOT NULL,
    pays              VARCHAR(60),
    contact            VARCHAR(120),
    telephone         VARCHAR(30),
    email             VARCHAR(120),

```

```

        incoterm          VARCHAR(20),
        actif             BOOLEAN NOT NULL
        DEFAULT TRUE
    );

CREATE TABLE commande_client (
    id                    SERIAL PRIMARY KEY,
    numero                VARCHAR(30) NOT
NULL UNIQUE,           -- CC-AAAA-NNNNN
    client_id            INT NOT NULL
REFERENCES client(id),
    produit_id           INT NOT NULL
REFERENCES produit(id),
    date_commande        DATE NOT NULL
DEFAULT CURRENT_DATE,
    quantite_commandee_kg NUMERIC(14,2) NOT
NULL CHECK (quantite_commandee_kg > 0),
    quantite_livree_kg    NUMERIC(14,2) NOT
NULL DEFAULT 0,
    destination          VARCHAR(150),
    incoterm              VARCHAR(20),
    date_livraison_prevue DATE,
    statut
statut_commande_client NOT NULL DEFAULT
'OUVERTE',
    observations          TEXT
);

CREATE TABLE expedition (
    id                    SERIAL PRIMARY KEY,
    numero                VARCHAR(30) NOT
NULL UNIQUE,           -- EXP-AAAA-NNNNN
    commande_id          INT REFERENCES
commande_client(id),

```

```

        client_id          INT NOT NULL
REFERENCES client(id),
        type              type_expedition NOT
NULL DEFAULT 'CONTENEUR_FLEXITANK',
        date_expedition    DATE NOT NULL
DEFAULT CURRENT_DATE,
        transporteur       VARCHAR(150),
        numero_bl          VARCHAR(40),
        destination        VARCHAR(150),
        statut             statut_validation
NOT NULL DEFAULT 'BROUILLON',
        total_litres       NUMERIC(14,2)
DEFAULT 0,
        total_kg           NUMERIC(14,2)
DEFAULT 0,
        observations       TEXT,
        created_at         TIMESTAMPTZ NOT
NULL DEFAULT now()
);

-- Empotage : une ligne par conteneur.
Numero de flexitank unique (RG-37).
CREATE TABLE chargement (
        id                 SERIAL PRIMARY KEY,
        expedition_id      INT NOT NULL
REFERENCES expedition(id) ON DELETE
CASCADE,
        tank_id            INT NOT NULL
REFERENCES tank(id),
        numero_conteneur   VARCHAR(20) NOT
NULL,
        numero_flexitank   VARCHAR(40) NOT
NULL UNIQUE, -- RG-37
        numero_remorque    VARCHAR(30),

```

```

        numero_tracteur    VARCHAR(30),
        chauffeur          VARCHAR(120),
        piece_chauffeur    VARCHAR(40),
        transporteur        VARCHAR(150),
        numero_scelle      VARCHAR(40),
        -- Volume et conversion
        quantite_litres     NUMERIC(14,2) NOT
NULL CHECK (quantite_litres > 0),
        temperature        NUMERIC(6,2),
        densite             NUMERIC(8,4),
        masse_calcullee_kg
NUMERIC(14,2),                -- litres x
densite
        -- Pesee pont bascule : fait foi
commerciallement (RG-38)
        poids_tare_kg       NUMERIC(14,2),
        poids_brut_kg       NUMERIC(14,2),
        poids_net_kg        NUMERIC(14,2),
        ecart_pesee_kg      NUMERIC(14,2)
GENERATED ALWAYS AS

        (COALESCE(poids_net_kg,0) -
COALESCE(masse_calcullee_kg,0)) STORED,
        -- Qualite et tracabilite
        bulletin_analyse_id INT,
        heure_debut         TIMESTAMPTZ,
        heure_fin           TIMESTAMPTZ,
        operateur_id        INT REFERENCES
utilisateur(id),
        statut              statut_validation
NOT NULL DEFAULT 'BROUILLON',
        observations        TEXT
);
ALTER TABLE tank_mouvement

```

```

    ADD CONSTRAINT fk_tm_chargement FOREIGN
KEY (chargement_id) REFERENCES
chargement(id);
CREATE INDEX idx_chargement_conteneur ON
chargement(numero_conteneur);
CREATE INDEX idx_chargement_flexitank ON
chargement(numero_flexitank);

-- Signatures apposees sur les documents
imprimables (RG-53)
CREATE TABLE signature_document (
    id                BIGSERIAL PRIMARY KEY,
    entite            VARCHAR(40) NOT NULL,
    entite_id        INT NOT NULL,
    ordre            SMALLINT NOT NULL,
    role_id          INT REFERENCES
role(id),
    utilisateur_id   INT REFERENCES
utilisateur(id),
    nom_signataire   VARCHAR(150),
    fonction          VARCHAR(120),
    date_signature   TIMESTAMPTZ,
    empreinte        VARCHAR(64),
    UNIQUE (entite, entite_id, ordre)
);

-- Mise a jour du stock tank et
interdiction du stock negatif (RG-40)
CREATE OR REPLACE FUNCTION
trg_tank_mouvement() RETURNS TRIGGER AS $$
DECLARE v_stock NUMERIC(14,2); v_delta
NUMERIC(14,2);
BEGIN
    SELECT stock_litres INTO v_stock FROM

```

```

tank WHERE id = NEW.tank_id FOR UPDATE;
    v_delta := CASE NEW.type_mvt
        WHEN 'ENTREE_PRODUCTION' THEN
NEW.quantite_litres
        WHEN 'TRANSFERT_ENTREE' THEN
NEW.quantite_litres
        WHEN 'AJUSTEMENT_JAUGE' THEN
NEW.quantite_litres -- peut etre negatif
        ELSE -NEW.quantite_litres END;
    IF v_stock + v_delta < 0 THEN
        RAISE EXCEPTION 'Stock insuffisant
dans le tank % (dispo: % l, demande: % l)',
            NEW.tank_id, v_stock,
NEW.quantite_litres;
    END IF;
    NEW.stock_avant_litres := v_stock;
    NEW.stock_apres_litres := v_stock +
v_delta;
    UPDATE tank SET stock_litres = v_stock
+ v_delta,
            stock_kg = (v_stock +
v_delta) * COALESCE(NEW.densite,
                    (SELECT
densite_reference FROM produit p
                                JOIN tank
t ON t.produit_id = p.id WHERE t.id =
NEW.tank_id), 1)
        WHERE id = NEW.tank_id;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER tg_tank_mouvement BEFORE
INSERT ON tank_mouvement
FOR EACH ROW EXECUTE FUNCTION

```

```

trg_tank_mouvement();

-- -----
-- -----
-- 16. DOMAINE LABORATOIRE ET QUALITE
-- -----
-- -----

CREATE TYPE type_valeur_analyse AS ENUM
('NUMERIQUE','TEXTE','LISTE','BOOLEEN');
CREATE TYPE conclusion_bulletin AS ENUM
('CONFORME','NON_CONFORME','DEROGATION','EN
_COURS');

-- Parametres entierement administrables
(RG-43) : la liste des analyses
-- physico-chimiques est une donnee, jamais
du code.
CREATE TABLE parametre_analyse (
    id                SERIAL PRIMARY KEY,
    code              VARCHAR(30) NOT NULL
UNIQUE,
    libelle           VARCHAR(150) NOT NULL,
    unite             VARCHAR(20),
    methode           VARCHAR(120),
                                -- norme ou
methode de reference
    type_valeur       type_valeur_analyse NOT
NULL DEFAULT 'NUMERIQUE',
    valeurs_possibles JSONB,
    decimales         SMALLINT DEFAULT 2,
    ordre_affichage  SMALLINT DEFAULT 1,
    actif             BOOLEAN NOT NULL
DEFAULT TRUE
);

```

```

-- Specifications versionnees par periode
de validite (RG-44)
CREATE TABLE specification (
    id                SERIAL PRIMARY KEY,
    produit_id        INT NOT NULL REFERENCES
produit(id),
    parametre_id      INT NOT NULL REFERENCES
parametre_analyse(id),
    client_id         INT REFERENCES
client(id),          -- NULL = specification
standard
    valeur_min        NUMERIC(14,4),
    valeur_cible      NUMERIC(14,4),
    valeur_max        NUMERIC(14,4),
    reference_norme    VARCHAR(120),
    obligatoire       BOOLEAN NOT NULL
DEFAULT TRUE,
    date_debut        DATE NOT NULL DEFAULT
CURRENT_DATE,
    date_fin          DATE,
    cree_par          INT REFERENCES
utilisateur(id)
);
CREATE INDEX idx_spec_produit ON
specification(produit_id, parametre_id,
date_debut);

CREATE TABLE point_prelevement (
    id                SERIAL PRIMARY KEY,
    code              VARCHAR(30) NOT NULL
UNIQUE,
    libelle           VARCHAR(150) NOT NULL,
    type              VARCHAR(30) NOT

```



```

NULL,          -- MP, PROCESS, TANK,
CHARGEMENT
    actif          BOOLEAN NOT NULL
DEFAULT TRUE
);

CREATE TABLE echantillon (
    id              SERIAL PRIMARY KEY,
    numero          VARCHAR(30) NOT
NULL UNIQUE,      -- ECH-AAAA-NNNNN
    client_uuid     UUID UNIQUE,
    date_prelevement  TIMESTAMPTZ NOT
NULL DEFAULT now(),
    point_id        INT REFERENCES
point_prelevement(id),
    produit_id      INT NOT NULL
REFERENCES produit(id),
    tank_id         INT REFERENCES
tank(id),
    journal_quart_id  INT REFERENCES
journal_quart(id),
    chargement_id    INT REFERENCES
chargement(id),
    preleve_par      INT REFERENCES
utilisateur(id),
    observation      TEXT
);

CREATE TABLE bulletin_analyse (
    id              SERIAL PRIMARY KEY,
    numero          VARCHAR(30) NOT
NULL UNIQUE,      -- BA-AAAA-NNNNN
    echantillon_id  INT NOT NULL
REFERENCES echantillon(id),

```

```

        saisi_par          INT REFERENCES
utilisateur(id),
        verifie_par       INT REFERENCES
utilisateur(id),
        approuve_par      INT REFERENCES
utilisateur(id),
        diffuse_a         INT REFERENCES
utilisateur(id),
        date_saisie       TIMESTAMPTZ,
        date_approbation  TIMESTAMPTZ,
        conclusion        conclusion_bulletin
NOT NULL DEFAULT 'EN_COURS',
        statut            statut_validation
NOT NULL DEFAULT 'BROUILLON',
        bulletin_rectifie_id INT REFERENCES
bulletin_analyse(id), -- RG-47
        commentaire      TEXT
);
ALTER TABLE chargement
    ADD CONSTRAINT fk_chargement_bulletin
FOREIGN KEY (bulletin_analyse_id)
REFERENCES bulletin_analyse(id);

CREATE TABLE analyse (
    id          SERIAL PRIMARY KEY,
    echantillon_id INT NOT NULL
REFERENCES echantillon(id) ON DELETE
CASCADE,
    parametre_id INT NOT NULL
REFERENCES parametre_analyse(id),
    valeur_numerique NUMERIC(14,4),
    valeur_texte     VARCHAR(120),
    specification_id INT REFERENCES
specification(id), -- spec appliquee

```

```

        conforme
        BOOLEAN,                                --
        calcule (RG-45)
        analyse_par          INT REFERENCES
utilisateur(id),
        date_analyse        TIMESTAMPTZ NOT
NULL DEFAULT now(),
        appareil            VARCHAR(80),
        observation         TEXT,
        UNIQUE (echantillon_id, parametre_id)
    );

CREATE TABLE non_conformite (
    id          SERIAL PRIMARY KEY,
    numero      VARCHAR(30) NOT NULL
    UNIQUE,
    bulletin_id INT REFERENCES
bulletin_analyse(id),
    produit_id  INT REFERENCES
produit(id),
    tank_id     INT REFERENCES
tank(id),
    journal_quart_id INT REFERENCES
journal_quart(id),
    description TEXT NOT NULL,
    decision
    VARCHAR(30),                                -- BLOCAGE,
    DECLASSEMENT, DEROGATION
    justification TEXT,
    traite_par    INT REFERENCES
utilisateur(id),
    date_ouverture TIMESTAMPTZ NOT NULL
    DEFAULT now(),
    date_cloture  TIMESTAMPTZ,

```

```

        statut          VARCHAR(20) NOT NULL
        DEFAULT 'OUVERTE'
    );

    -- Conformite calculee automatiquement
    -- contre la specification en vigueur
    -- a la date du prelevement (RG-45)
    CREATE OR REPLACE FUNCTION
    trg_conformite_analyse() RETURNS TRIGGER AS
    $$
    DECLARE s RECORD; v_date DATE; v_prod INT;
    v_client INT;
    BEGIN
        SELECT e.date_prelevement::date,
        e.produit_id,
        (SELECT c.client_id FROM
        chargement ch
        JOIN expedition ex ON ex.id =
        ch.expedition_id
        JOIN commande_client c ON
        c.id = ex.commande_id
        WHERE ch.id = e.chargement_id)
        INTO v_date, v_prod, v_client
        FROM echantillon e WHERE e.id =
        NEW.echantillon_id;

        SELECT * INTO s FROM specification
        WHERE produit_id = v_prod AND
        parametre_id = NEW.parametre_id
        AND (client_id = v_client OR
        client_id IS NULL)
        AND date_debut <= v_date AND
        (date_fin IS NULL OR date_fin >= v_date)
        ORDER BY client_id NULLS LAST,

```

```

date_debut DESC LIMIT 1;

        IF s.id IS NOT NULL AND
NEW.valeur_numerique IS NOT NULL THEN
            NEW.specification_id := s.id;
            NEW.conforme := (s.valeur_min IS
NULL OR NEW.valeur_numerique >=
s.valeur_min)
                                AND (s.valeur_max IS
NULL OR NEW.valeur_numerique <=
s.valeur_max);
        END IF;
        RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER tg_conformite_analyse BEFORE
INSERT OR UPDATE ON analyse
FOR EACH ROW EXECUTE FUNCTION
trg_conformite_analyse();

-- -----
-----
-- 17. VUES DE PILOTAGE DES NOUVEAUX
DOMAINES
-- -----
-----

CREATE VIEW v_bilan_quart AS
SELECT jq.numero, jq.date_journee,
jq.quart, l.libelle AS ligne,
        u.nom || ' ' ||
COALESCE(u.prenom, '') AS chef_quart,
        jq.total_entrees_kg,
jq.total_sorties_kg, jq.ecart_kg,
jq.ecart_pct,

```

```

jq.rendement_pct,
jq.heures_fonctionnement,
CASE WHEN jq.heures_fonctionnement >
0
THEN ROUND((jq.total_entrees_kg
/ jq.heures_fonctionnement)::numeric,2) END
AS debit_kg_h,
(SELECT COALESCE(SUM(duree_min),0)
FROM journal_arret WHERE journal_quart_id =
jq.id) AS minutes_arret,
jq.statut
FROM journal_quart jq
JOIN ligne_production l ON l.id =
jq.ligne_id
JOIN utilisateur u ON u.id =
jq.chef_quart_id
ORDER BY jq.date_journee DESC, jq.quart;

CREATE VIEW v_repartition_sorties AS
SELECT jq.date_journee, p.libelle AS
produit, p.type,
SUM(s.quantite_kg) AS quantite_kg,
ROUND((SUM(s.quantite_kg) /
NULLIF(jq.total_entrees_kg,0) *
100)::numeric, 2) AS taux_pct
FROM journal_sortie s
JOIN journal_quart jq ON jq.id =
s.journal_quart_id
JOIN produit p ON p.id = s.produit_id
WHERE jq.statut IN ('APPROUVE','DIFFUSE')
GROUP BY jq.date_journee, p.libelle,
p.type, jq.total_entrees_kg;

CREATE VIEW v_stock_tanks AS

```

```

SELECT t.code, t.libelle, p.libelle AS
produit,
       t.stock_litres, t.stock_kg,
t.litres_reserves,
       (t.stock_litres - t.litres_reserves)
AS disponible_litres,
       t.capacite_litres,
       ROUND((t.stock_litres /
NULLIF(t.capacite_litres,0) *
100)::numeric,1) AS taux_remplissage_pct,
       (SELECT j.ecart_pct FROM jaugeage j
WHERE j.tank_id = t.id
       ORDER BY j.date_jaugeage DESC
LIMIT 1) AS dernier_ecart_jauge_pct
FROM tank t LEFT JOIN produit p ON p.id =
t.produit_id
WHERE t.actif;

```

```

-- Tracabilite descendante : d'un conteneur
vers son origine (RG-41)
CREATE VIEW v_tracabilite_chargement AS
SELECT ch.numero_conteneur,
ch.numero_flexitank, ch.numero_scelle,
       ch.numero_remorque,
ch.numero_tracteur, ch.chauffeur,
       ch.quantite_litres, ch.temperature,
ch.densite,
       ch.masse_calculee_kg,
ch.poids_net_kg, ch.ecart_pesee_kg,
       t.code AS tank_source, ex.numero AS
expedition, cl.raison_sociale AS client,
       ba.numero AS bulletin_analyse,
ba.conclusion,
       (SELECT string_agg(DISTINCT

```

```

jq.numero, ', '))
        FROM tank_mouvement tm
        JOIN journal_quart jq ON jq.id =
tm.journal_quart_id
        WHERE tm.tank_id = ch.tank_id
        AND tm.type_mvt =
'ENTREE_PRODUCTION'
        AND tm.date_mvt <=
ch.heure_debut
        AND tm.date_mvt >=
ch.heure_debut - interval '30 days') AS
lots_production
FROM chargement ch
JOIN tank t ON t.id = ch.tank_id
JOIN expedition ex ON ex.id =
ch.expedition_id
JOIN client cl ON cl.id = ex.client_id
LEFT JOIN bulletin_analyse ba ON ba.id =
ch.bulletin_analyse_id;

CREATE VIEW v_conformite_laboratoire AS
SELECT date_trunc('month',
e.date_prelevement) AS mois,
        p.libelle AS produit,
        COUNT(*) AS nb_analyses,
        COUNT(*) FILTER (WHERE a.conforme)
AS nb_conformes,
        ROUND((COUNT(*) FILTER (WHERE
a.conforme)::numeric / NULLIF(COUNT(*),0) *
100), 1) AS taux_conformite_pct
FROM analyse a
JOIN echantillon e ON e.id =
a.echantillon_id
JOIN produit p ON p.id = e.produit_id

```



```
GROUP BY 1, 2 ORDER BY 1 DESC;
```

```
CREATE VIEW v_documents_en_attente AS
SELECT 'RAPPORT_QUART' AS type_document,
numero, statut::text,
        updated_at AS depuis, (now() -
updated_at) AS anciennete
FROM journal_quart WHERE statut IN
('SOUMIS','VERIFIE')
UNION ALL
SELECT 'BULLETIN_ANALYSE', numero,
statut::text, date_saisie, (now() -
date_saisie)
FROM bulletin_analyse WHERE statut IN
('SOUMIS','VERIFIE')
UNION ALL
SELECT 'EXPEDITION', numero, statut::text,
created_at, (now() - created_at)
FROM expedition WHERE statut IN
('SOUMIS','VERIFIE')
ORDER BY anciennete DESC;
```

```
-- -----
-----
-- 18. PARAMETRES ET DONNEES INITIALES DES
NOUVEAUX DOMAINES
-- -----
-----
```

```
INSERT INTO parametre (cle, valeur,
description) VALUES
('SEUILALERTEECARTBILANPCT','1','Ecart
de bilan matiere declenchant un commentaire
obligatoire'),
```

```

('SEUIL_BLOPAGE_ECART_BILAN_PCT','3','Ecart
de bilan matiere bloquant la soumission du
rapport'),
('DUREE_ARRET_GENERANT_DI_MIN','30','Duree
d''arret machine generant une demande
d''intervention (RG-35)'),

('TOLERANCE_ECART_PESEE_PCT','0.5','Toleran
ce entre masse calculee et masse pesee au
pont bascule'),
('DELAIRELANCE_VALIDATION_H','24','Delai
avant relance automatique d''un valideur
(RG-55)'),
('FORMAT_NUM_JQ','JQ-{AAAA}-
{00000}','Format de numerotation des
rapports de quart'),
('FORMAT_NUM_EXP','EXP-{AAAA}-
{00000}','Format de numerotation des
expeditions'),
('FORMAT_NUM_BA','BA-{AAAA}-
{00000}','Format de numerotation des
bulletins d''analyse');

INSERT INTO role (code, libelle) VALUES
('MAGASIN_MP','Magasinier matiere
premiere'),
('CHEF_QUART','Chef de quart'),
('RESP_PRODUCTION','Responsable
production'),
('CHEF_USINE','Chef d''usine / directeur
d''usine'),
('AGENT_EXPEDITION','Agent
d''expedition'),
('TECH_LABO','Technicien de laboratoire'),

```

```
 ('ASSIST_RESP_LABO','Assistant du
responsable laboratoire'),
 ('RESP_LABO','Responsable laboratoire'),
 ('RESP_QUALITE','Responsable qualite'),
 ('DIRECTION_GENERALE','Direction generale
/ PDG');
```

```
INSERT INTO produit (code, libelle, type,
unite) VALUES
 ('MP-001','Matiere
premiere','MATIERE_PREMIERE','kg'),
 ('PF-001','Produit
fini','PRODUIT_FINI','kg'),
 ('SP-TOU','Tourteau (apres
pressage)','SOUS_PRODUIT','kg'),
 ('SP-POU','Poussiere (apres
tamisage)','SOUS_PRODUIT','kg'),
 ('SP-COQ','Coque','SOUS_PRODUIT','kg');
```

```
INSERT INTO point_prelevement (code,
libelle, type) VALUES
 ('PP-MP','Reception matiere
premiere','MP'),
 ('PP-PROC','Cours de
fabrication','PROCESS'),
 ('PP-TANK','Tank de stockage','TANK'),
 ('PP-CHAR','Chargement /
empotage','CHARGEMENT');
```

```
-- Circuits de validation initiaux
(parametrables ensuite dans l'interface)
INSERT INTO circuit_validation (code,
entite, libelle) VALUES
 ('RAPPORT_QUART','journal_quart','Rapport
```

```
de quart de production'),

('BULLETIN_ANALYSE','bulletin_analyse','Bulletin d\'analyse laboratoire'),
('EXPEDITION','expedition','Expedition et bon de chargement'),

('DEMANDE_MATIERE','demande_matiere','Demande de de matiere premiere');
--
=====
=====
-- FIN DE L'EXTENSION V2.0
--
=====
=====
```